Finalizers and Generic Bindings

Rohit Goswami · 2026-08-10 · python · fortran · numpy · f2py · ramblings

> *Cleanup should never be the programmer's problem, and the right interface hides exactly the right amount of detail. Fortran's finalizers and generic bindings are built on these two convictions.*

Background

Last time we dealt with pointer members and deferred-length character strings. This time we have two features from the type-bound procedure side of F2018: finalizers and generic bindings.

Both sit in the `CONTAINS` block of a derived type definition. That matters because `crackfortran` ignores `CONTAINS` entirely at the type level[^fn:1]. Everything in this post relies on scanning the original Fortran source directly.

The code corresponds to the post/f2py-finalizers-generics tag.

Finalizers

What finalization means

F2018 7.5.6 defines finalization as automatic cleanup that runs when a finalizable entity ceases to exist. The standard lays out a three-step process (F2018 7.5.6.2):

1. If the entity's type has a matching `FINAL` subroutine, call it.
2. Finalize each finalizable component of the entity.
3. If the entity's parent type is finalizable, finalize the parent component.

F2018 7.5.6.3 paragraph 2 says that `DEALLOCATE` on a finalizable entity triggers finalization. The opaque pointer destructor already calls `deallocate`, so finalization happens with no extra work at the wrapper level.

The Fortran type

```
module finalizer_mod
  use iso_c_binding
  implicit none
  integer :: finalize_count = 0

  type :: Tracked
    integer :: value
  contains
    final :: tracked_finalize
  end type Tracked
contains
  subroutine tracked_finalize(self)
    type(Tracked), intent(inout) :: self
    finalize_count = finalize_count + 1
  end subroutine
  function get_finalize_count() result(count)
    integer :: count
    count = finalize_count
  end function
  subroutine reset_finalize_count()
    finalize_count = 0
  end subroutine
end module
```

`Tracked` increments a module-level counter each time an instance is finalized.

Manual Fortran wrappers

The wrappers are the same create/destroy pattern. The important thing is the destructor:

```
module tracked_wrappers
  use finalizer_mod
  use iso_c_binding
  implicit none
contains
  function tracked_create(val) result(cptr) bind(c)
    integer(c_int), value :: val
    type(c_ptr) :: cptr
    type(Tracked), pointer :: obj
    allocate(obj)
    obj%value = val
    cptr = c_loc(obj)
  end function

  subroutine tracked_destroy(cptr) bind(c)
    type(c_ptr), value :: cptr
    type(Tracked), pointer :: obj
    call c_f_pointer(cptr, obj)
    deallocate(obj)  ! triggers tracked_finalize automatically
  end subroutine

  function tracked_get_count() result(n) bind(c)
    integer(c_int) :: n
    n = get_finalize_count()
  end function

  subroutine tracked_reset_count() bind(c)
    call reset_finalize_count()
  end subroutine
end module
```

No code calls `tracked_finalize` directly. The `deallocate(obj)` in the destructor is enough. Since `Tracked` declares a `FINAL` subroutine, the Fortran runtime calls `tracked_finalize` before releasing the memory. The standard does the work.

Testing from C

▶ C test (click to expand)